SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INTERACTIVE TREE SIMULATOR

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	**Assessment:**	Assessment Tool 1 – Interactive Tree Simulator
Weightage:	30% of CIA	**Total Marks:**	100
CO3 Statement:	*Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.*		
Student Name:	B.Sathvika	**Reg. No.:**	192525224
Section / Batch:	2nd year AIML	**Date:**	27/07/2026

Code of Conduct

I B.Sathvika(192525224)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.Sathvika

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic	Focus	Q Nos	Marks
Binary Search Tree	Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals	Traversal types, In order, Post order and Pre order	2	20
AVL Tree	Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree	Properties, structures and Operations	4	20
Red Black Tree	Properties, Example and Operations	5	20
	GRAND TOTAL:	**100 Marks**	

Annexure A – Interactive Tree Simulator

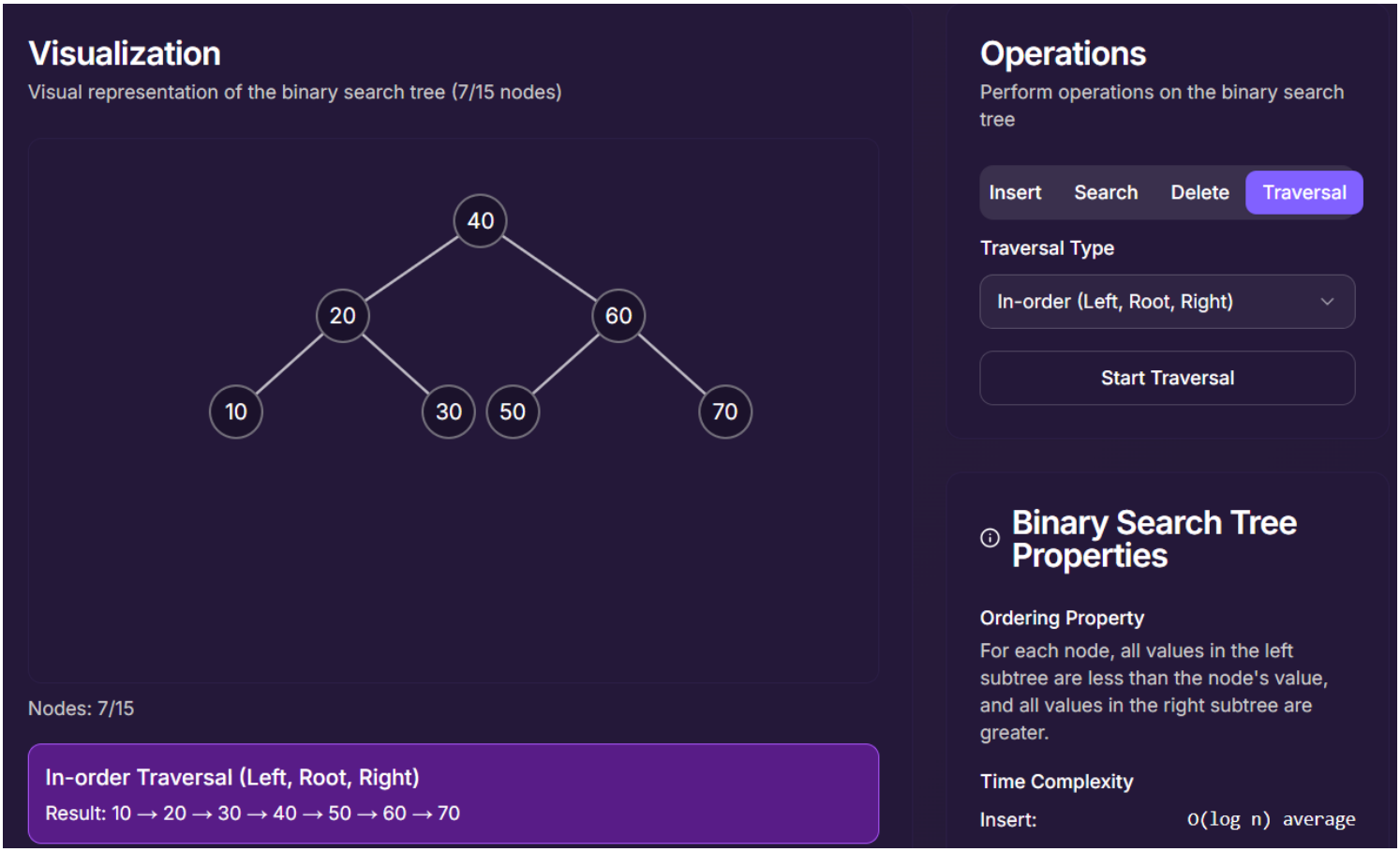
1. Construct a Binary Search Tree (BST) by inserting the following keys: 40, 20, 10, 30, 60, 50, 70
   Perform:
   a. Inorder Traversal
   b. Search for 70
   c. Delete 50
   d. Display the final BST.
2. Complete Tree Traversals by Inserting 55,35,75,25,45,65,85. Find all the traversals.
   a. Inorder
   b. Preorder
   c. Postorder
3. Insert 40, 20, 60, 10, 30, 50, 70, 5 into an AVL Tree. Determine the balance factor of every node before and after balancing.
4. Construct a B-Tree of order 3 by inserting 10, 20, 30, 40, 50.
5. Construct a Red-Black Tree using 50, 40, 60, 30, 45, 55, 70, 20. Delete 30 and analyze how Red-Black properties are restored.

Rubrics for Calculation

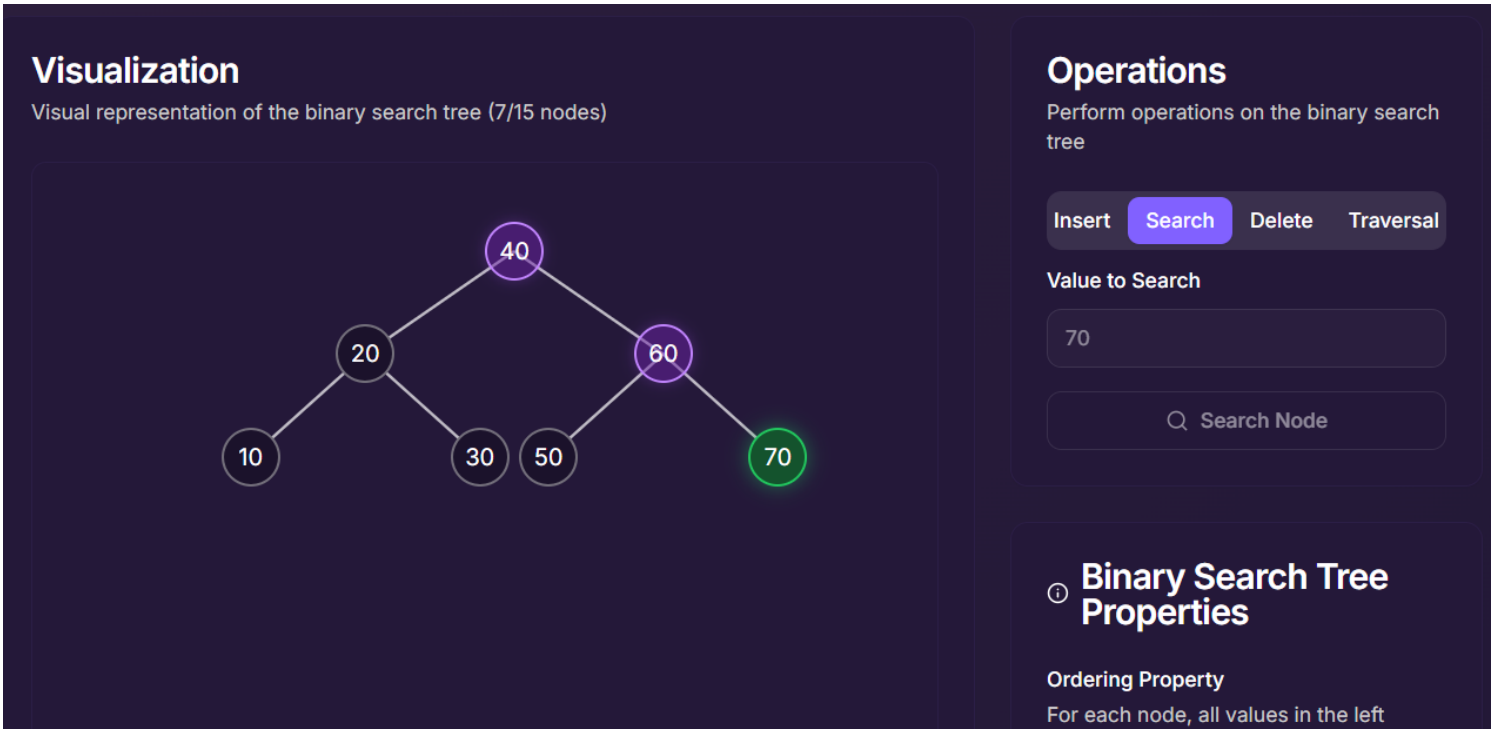
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations
Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, node split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

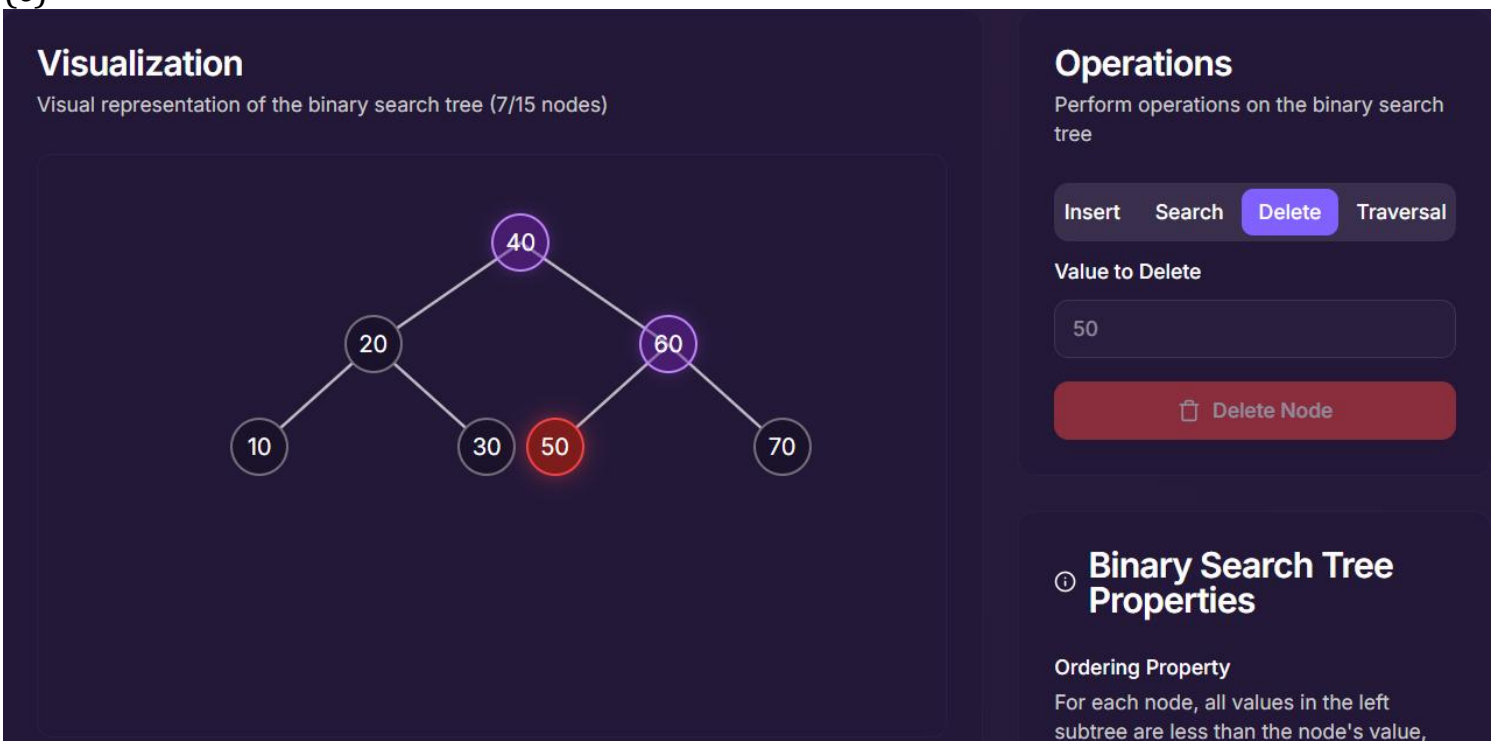
1}
(a)


Visualization
Visual representation of the binary search tree (7/15 nodes)
40
20
60
10
30
50
70
Nodes: 7/15
In-order Traversal (Left, Root, Right)
Result: 10 → 20 → 30 → 40 → 50 → 60 → 70
Operations
Perform operations on the binary search tree
Insert
Search
Delete
Traversal
Traversal Type
In-order (Left, Root, Right)
Start Traversal
Binary Search Tree Properties
Ordering Property
For each node, all values in the left subtree are less than the node's value, and all values in the right subtree are greater.
Time Complexity
Insert:
O(log n) average

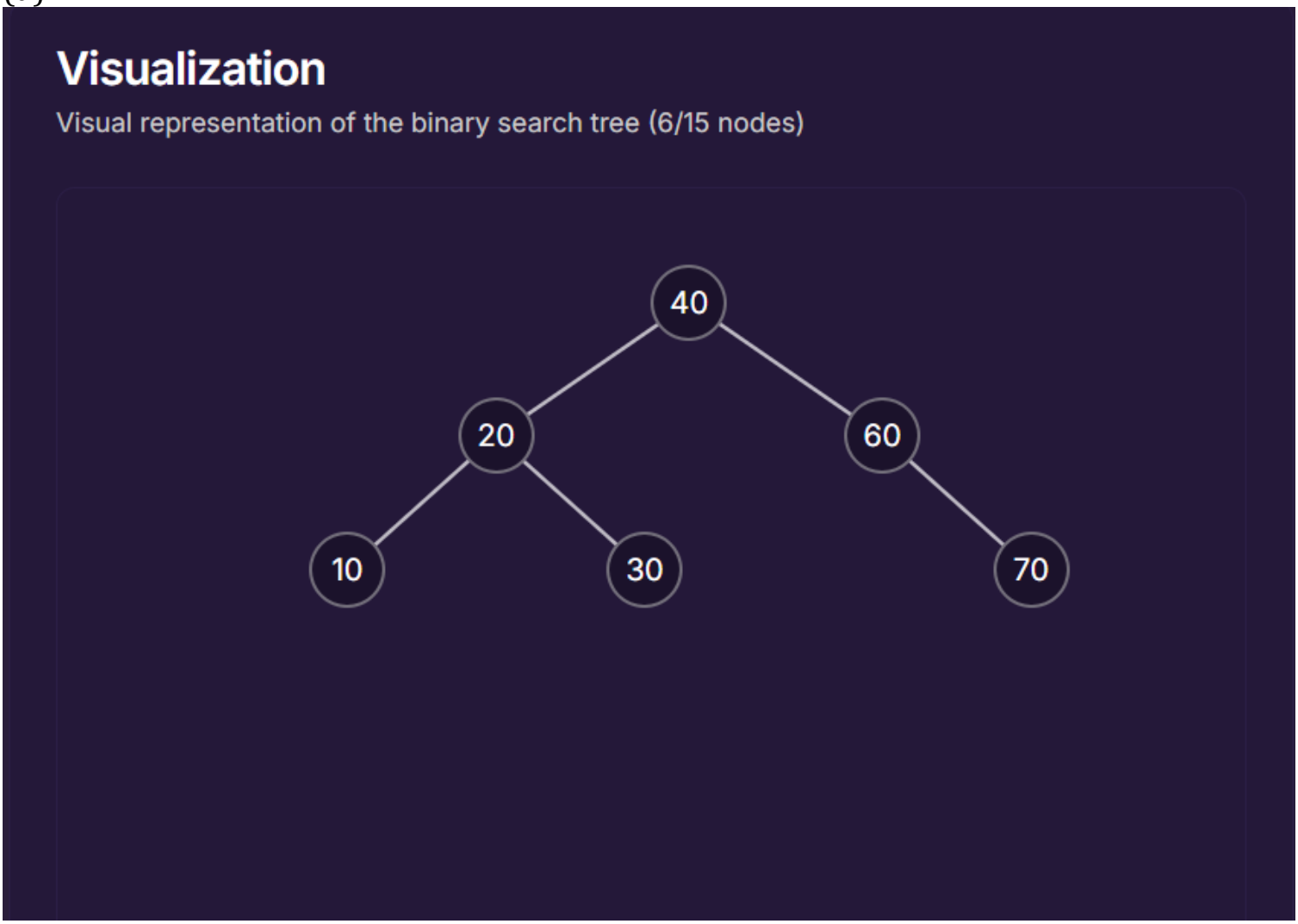
(b)


Visualization
Visual representation of the binary search tree (7/15 nodes)
40
20
60
10
30
50
70
Operations
Perform operations on the binary search tree
Insert
Search
Delete
Traversal
Value to Search
70
Search Node
Binary Search Tree Properties
Ordering Property
For each node, all values in the left

(c)


Visualization
Visual representation of the binary search tree (7/15 nodes)
40
20
60
10
30
50
70
Operations
Perform operations on the binary search tree
Insert
Search
Delete
Traversal
Value to Delete
50
Delete Node
Binary Search Tree Properties
Ordering Property
For each node, all values in the left subtree are less than the node's value,

(d)


Visualization
Visual representation of the binary search tree (6/15 nodes)
40
20
60
10
30
70

2)

(a)


Visualization
Visual representation of the binary search tree (7/15 nodes)
55
35
75
25
45
65
85
Nodes: 7/15
In-order Traversal (Left, Root, Right)
Result: 25 → 35 → 45 → 55 → 65 → 75 → 85

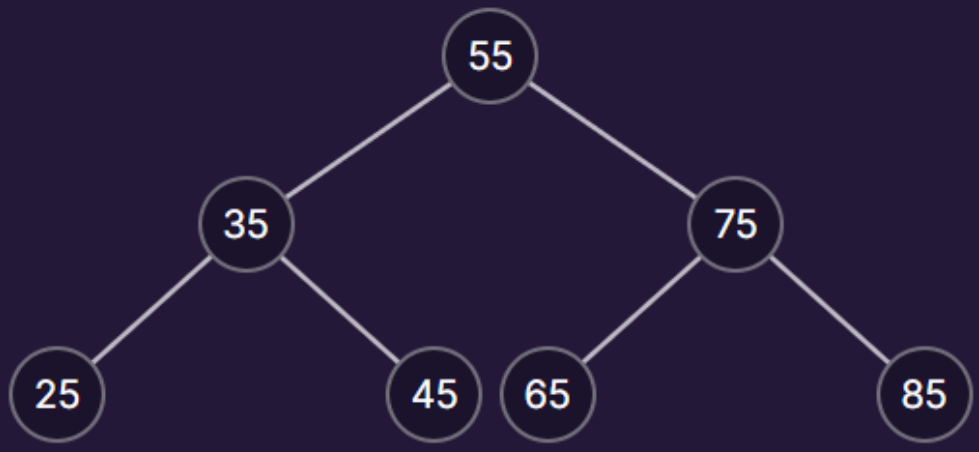
(b)


Visualization
Visual representation of the binary search tree (7/15 nodes)
55
35
75
25
45
65
85
Nodes: 7/15
Pre-order Traversal (Root, Left, Right)
Result: 55 → 35 → 25 → 45 → 75 → 65 → 85

(c)

Visualization

Visual representation of the binary search tree (7/15 nodes)

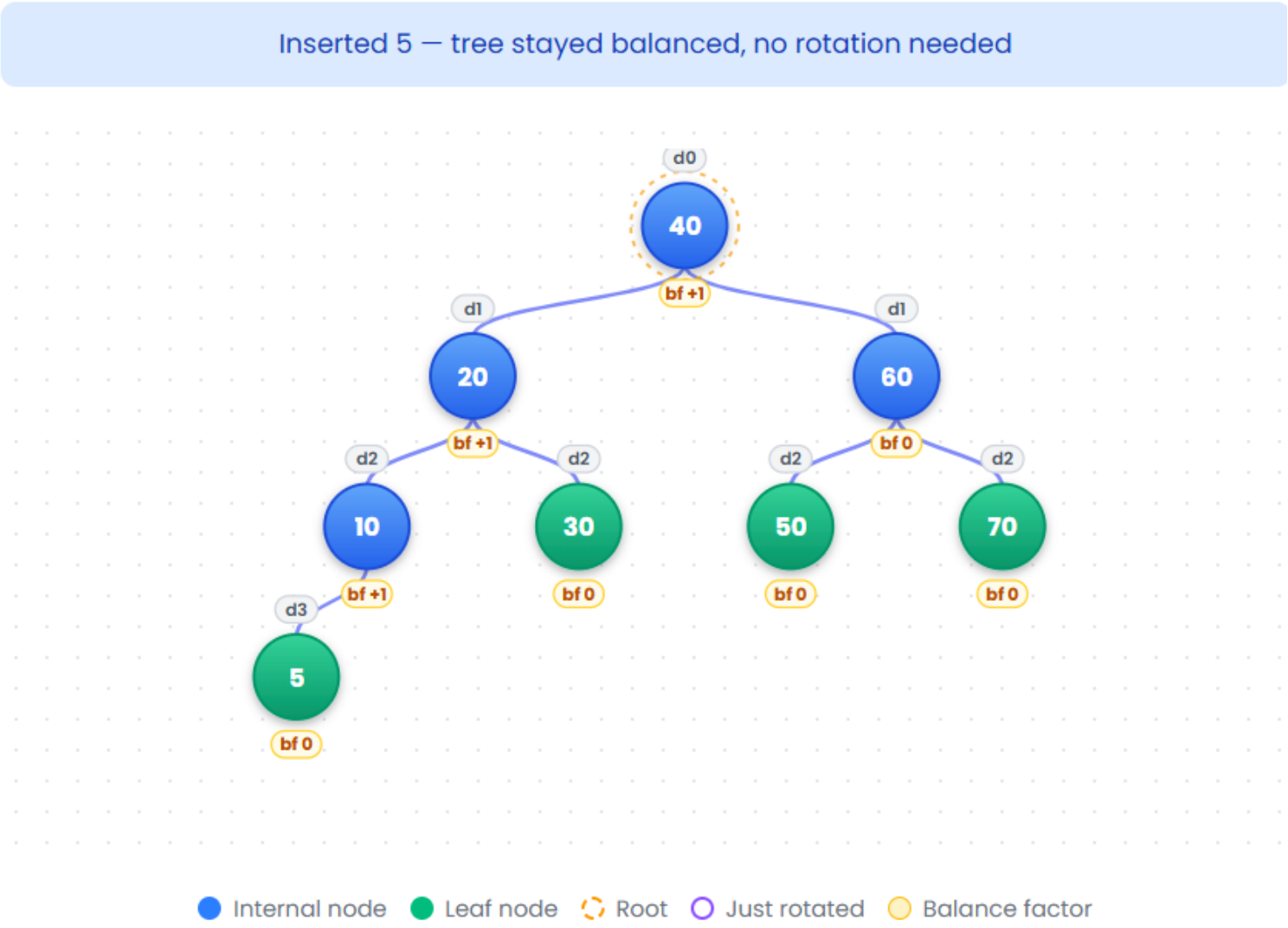


Nodes: 7/15

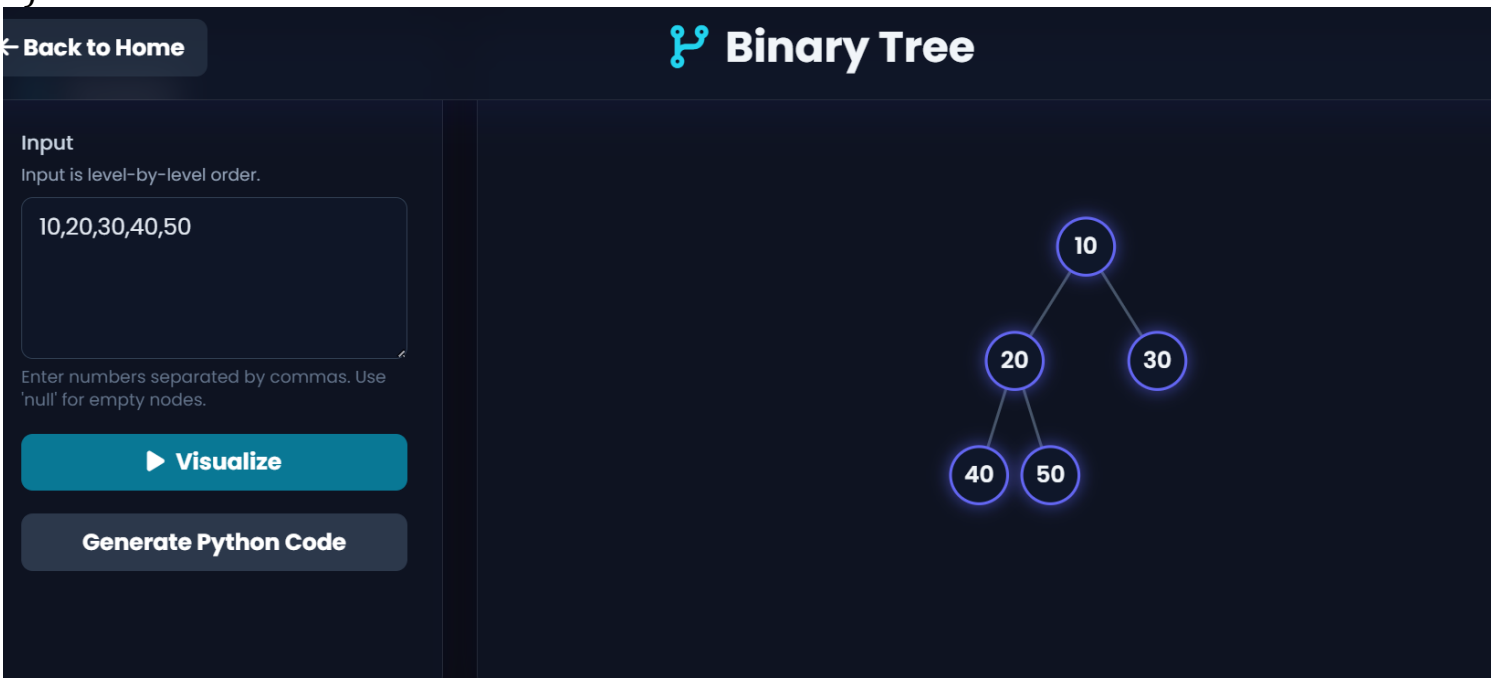
Post-order Traversal (Left, Right, Root)

Result: 25 → 45 → 35 → 65 → 85 → 75 → 55

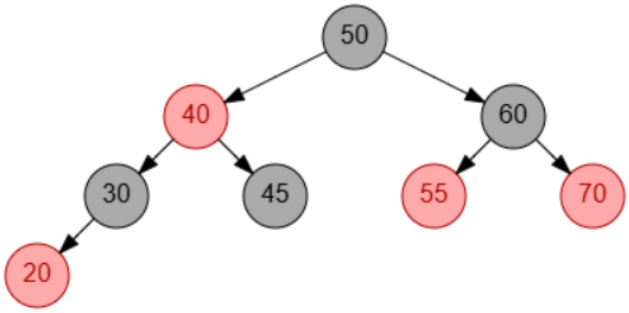
3)


Inserted 5 — tree stayed balanced, no rotation needed
d0
40
bf +1
d1
d1
20
60
bf +1
bf 0
d2
d2
d2
d2
10
30
50
70
bf +1
bf 0
bf 0
bf 0
d3
5
bf 0
Internal node
Leaf node
Root
Just rotated
Balance factor

4)

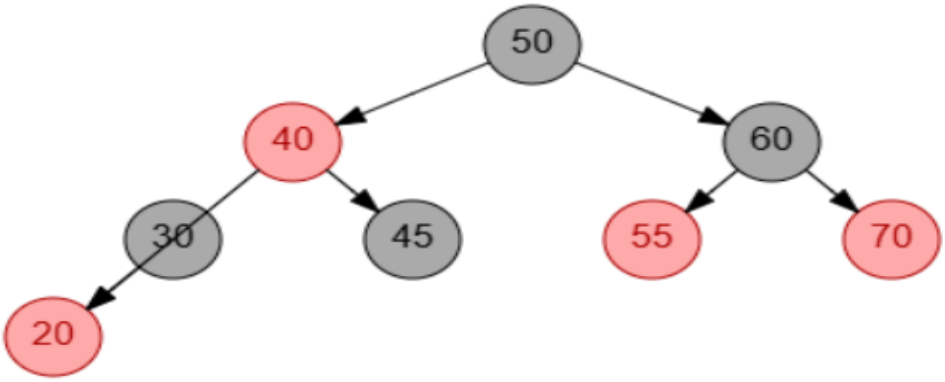

Back to Home
Binary Tree
Input
Input is level-by-level order.
10,20,30,40,50
Enter numbers separated by commas. Use 'null' for empty nodes.
Visualize
Generate Python Code
10
20
30
40
50

5)



Observations

Node to delete has no right child.
Set parent of deleted node to left child of deleted node.



Final red black tree: